

UNIT III: CONCURRENCY CONTROL & DEADLOCKS

Continuous Concurrent Evaluation (CCE2) — Comprehensive Worked Solutions

Course Pattern: TE IT - 2024 Pattern (SPPU)
Document Type: CCE2 Model Answer Solutions

Academic Term: Semester V (2026)
Course Code: PCC-301-ITT

NOTE:

This document provides exhaustive, publication-grade model answers and worked solutions for all questions in **Unit III (Concurrency Control & Deadlocks)** from the official CCE2 question bank. Step-by-step semaphore trace explanations, resource allocation graphs, Banker's algorithm matrices, and prevention strategies are fully worked out.

Q1. Producer-Consumer Problem (E-Commerce Warehouse)

SCENARIO:

An e-commerce warehouse has a shared order buffer with a capacity of **5 orders**. Multiple producer threads add orders to the buffer and consumer threads remove orders.

Part (a) — Identify Semaphores Required for Synchronization

To synchronize multiple producer and consumer threads accessing a finite bounded buffer of size $N = 5$, three semaphores are required:

1. **mutex (Binary Semaphore / Mutex Lock):**

- **Initial Value:** 1
- **Purpose:** Provides **mutual exclusion** when accessing the shared order buffer critical section. It ensures that only one thread (either a producer or a consumer) modifies buffer pointers and writes/reads data at any given instant.

2. **empty (Counting Semaphore):**

- **Initial Value:** 5 (Buffer capacity $N = 5$)
- **Purpose:** Tracks the number of **empty slots** currently available in the warehouse order buffer. A producer decrements **empty** before inserting an order; if **empty** == 0, the producer blocks (waits) until a consumer frees up a slot.

3. **full (Counting Semaphore):**

- **Initial Value:** 0
- **Purpose:** Tracks the number of **occupied slots** containing valid orders ready for processing. A consumer decrements **full** before fetching an order; if **full** == 0, the consumer blocks (waits) until a producer adds a new order.

Part (b) — Pseudocode for Producer and Consumer using Semaphores

```
// Shared Global Variables & Synchronization Primitives
#define BUFFER_SIZE 5
Order buffer[BUFFER_SIZE];
int in = 0;          // Index to add next order
int out = 0;         // Index to remove next order

semaphore mutex = 1; // Controls mutual exclusion to buffer
semaphore empty = 5; // Tracks available vacant slots
semaphore full = 0;  // Tracks filled slots ready for consumption

// — PRODUCER THREAD —
void* producer(void* param) {
    Order new_order;
    while (1) {
        new_order = produce_order(); // Generate an e-commerce order

        wait(empty);                  // Wait for an empty slot (decrement empty)
        wait(mutex);                  // Enter critical section

        // Critical Section: Add order to shared buffer
        buffer[in] = new_order;
        in = (in + 1) % BUFFER_SIZE;

        signal(mutex);                // Exit critical section
        signal(full);                 // Increment count of filled slots
    }
}

// — CONSUMER THREAD —
void* consumer(void* param) {
    Order order_to_process;
    while (1) {
        wait(full);                   // Wait for a filled slot (decrement full)
        wait(mutex);                  // Enter critical section

        // Critical Section: Remove order from shared buffer
        order_to_process = buffer[out];
        out = (out + 1) % BUFFER_SIZE;

        signal(mutex);                // Exit critical section
        signal(empty);                // Increment count of empty slots

        process_order(order_to_process); // Dispatch/fulfill order
    }
}
```

Key Implementation Rule: The order of wait() calls is critical: wait(empty) must precede wait(mutex) in the producer, and wait(full) must precede wait(mutex) in the consumer. Inverting these would cause a **deadlock** if the buffer is full and a producer acquires mutex.

Q2. Deadlock Scenario in E-Commerce System

SCENARIO:

Consider the following situation in an e-commerce system:

- **P_1**: Currently Holding **R_1 (Database Lock)**, Waiting For **R_2 (Inventory Lock)**
- **P_2**: Currently Holding **R_2 (Inventory Lock)**, Waiting For **R_3 (Payment Lock)**
- **P_3**: Currently Holding **R_3 (Payment Lock)**, Waiting For **R_1 (Database Lock)**

Part (a) — Draw the Resource Allocation Graph (RAG)

In a Resource Allocation Graph (RAG), processes are represented by circles, resources by rectangles/squares, assignment edges by arrows from Resource to Process ($R_j \rightarrow P_i$), and request edges by arrows from Process to Resource ($P_i \rightarrow R_j$).

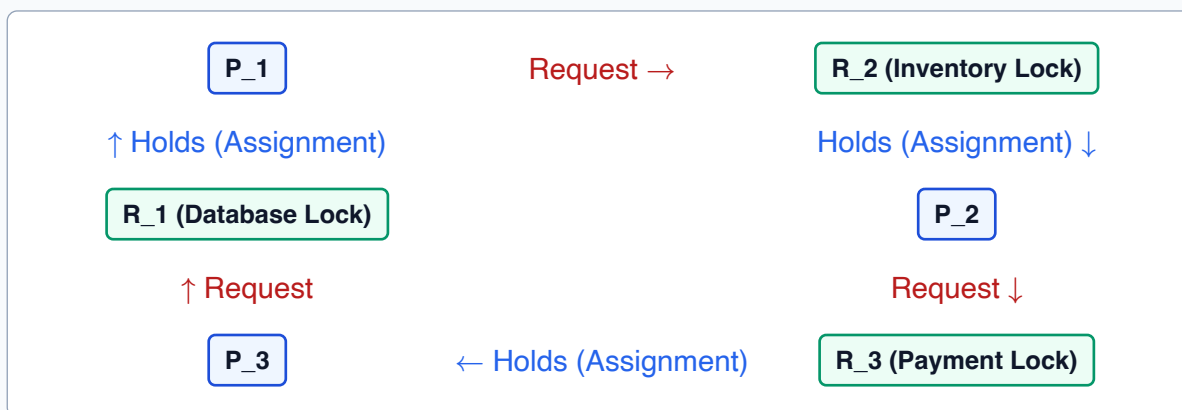


Figure 3.1: Resource Allocation Graph (RAG) Showing Circular Wait

Edge Summary:

- **Assignment Edges:** $R_1 \rightarrow P_1$, $R_2 \rightarrow P_2$, $R_3 \rightarrow P_3$
- **Request Edges:** $P_1 \rightarrow R_2$, $P_2 \rightarrow R_3$, $P_3 \rightarrow R_1$

Part (b) — Determine Whether System is in Deadlock (Justification)

Conclusion: YES, the system is in DEADLOCK.

Formal Justification:

1. **Single-Instance Resources:** Each lock (R_1, R_2, R_3) has exactly **one instance**. In a single-instance resource graph, the existence of a cycle is both a **necessary and sufficient condition** for deadlock.
2. **Directed Cycle:** The graph contains a closed directed cycle:

$$P_1 \rightarrow R_2 \rightarrow P_2 \rightarrow R_3 \rightarrow P_3 \rightarrow R_1 \rightarrow P_1$$

3. **Verification of Coffman's Four Conditions:**

- **Mutual Exclusion:** Locks (R_1, R_2, R_3) are exclusive and held non-shareably.
- **Hold and Wait:** P_1 holds R_1 while waiting for R_2 ; P_2 holds R_2 while waiting for R_3 ; P_3 holds R_3 while waiting for R_1 .
- **No Preemption:** An allocated lock cannot be forcefully taken away from a process.
- **Circular Wait:** A closed circular chain of processes exists where each process waits for a resource held by the next process in the cycle.

All four Coffman conditions hold simultaneously, confirming a permanent deadlock state.

Part (c) — Method to Prevent This Deadlock

Recommended Method: Strict Global Resource Ordering (Havender's Linear Ordering)

- **Mechanism:** Impose a total linear order on all resource types by defining a one-to-one enumeration function $F : R \rightarrow \mathbb{N}$:

$$F(R_1) = 1 \quad (\text{Database Lock}), \quad F(R_2) = 2 \quad (\text{Inventory Lock}), \quad F(R_3) = 3 \quad (\text{Payment Lock})$$

- **Protocol Rule:** Any process requesting multiple resources must acquire them **strictly in increasing order of their numerical enumeration**. A process can request a resource R_j only if $F(R_j) > F(R_i)$ for all resources R_i it currently holds.
- **Resolution of the Scenario:** Under this protocol, process P_3 wanting both R_3 and R_1 is forbidden from holding R_3 ($F = 3$) and subsequently requesting R_1 ($F = 1$). It must acquire R_1 first and then R_3 . This mathematically breaks the **Circular Wait** condition, making deadlocks impossible.

Q3. Readers-Writers Problem (Student Result Database)

SCENARIO:

A student-result database is accessed by multiple users. Several users can read the result simultaneously, but when a faculty member updates a result, no other reader or writer should access the database.

Part (a) — Identify the Synchronization Problem

This scenario represents the classic **First Readers-Writers Problem** (Readers-preference problem with reader concurrency and writer mutual exclusion).

Part (b) — Why Mutual Exclusion is Required for Writers

Mutual exclusion is required for writers to prevent **Race Conditions** and ensure **Data Consistency (ACID Properties)**:

1. **Writer-Writer Conflict (Lost Update Problem)**: If two faculty members attempt to update a student's grade concurrently without mutual exclusion, one update will overwrite the other without incorporating its changes, causing lost updates.
2. **Reader-Writer Conflict (Dirty Read & Inconsistent State)**: While a writer is executing multiple update statements (e.g., updating marks, calculating total, recomputing CGPA), the database temporarily resides in an inconsistent intermediate state. If readers are allowed concurrent access, they will read corrupted or incomplete data (**dirty reads**).
3. Therefore, a writer requires **exclusive lock** access: no other reader and no other writer may enter the database while a write is ongoing.

Part (c) — Semaphore-Based Pseudocode for Reader and Writer

```
// Shared Variables and Synchronization Primitives
int read_count = 0;           // Number of readers currently reading
semaphore rw_mutex = 1;       // Common to both readers and writers (ensures exclusive writing)
semaphore mutex = 1;          // Protects read_count updates from race conditions

// — WRITER PROCESS (Faculty Updating Result) —————
void writer() {
    while (1) {
        wait(rw_mutex);       // Request exclusive access to database

        // — Critical Section: Write/Update Student Result —
        update_student_result();

        signal(rw_mutex);     // Release exclusive access
    }
}

// — READER PROCESS (Student/User Viewing Result) —————
void reader() {
    while (1) {
        wait(mutex);           // Mutual exclusion for updating read_count
        read_count++;
        if (read_count == 1) {
            wait(rw_mutex);    // First reader locks out all writers
        }
        signal(mutex);

        // — Reading Section: Concurrent Read Access —————
        read_student_result();

        wait(mutex);           // Mutual exclusion for updating read_count
        read_count--;
        if (read_count == 0) {
            signal(rw_mutex);  // Last reader unlocks database for writers
        }
        signal(mutex);
    }
}
```

Q4. Banker's Algorithm for Deadlock Avoidance

SCENARIO:

A system has three resource types R_1, R_2, R_3 . Available resources: Available = (3, 2, 2)

Process	Allocation (R ₁ R ₂ R ₃)	Maximum (R ₁ R ₂ R ₃)
P ₁	1 0 1	2 1 1
P ₂	1 1 0	2 2 2
P ₃	1 0 1	3 1 2

Table 4.1: Current Allocation and Maximum Demand Matrices

Part (a) — Calculate the Need Matrix

Formula: $\text{Need}[i][j] = \text{Maximum}[i][j] - \text{Allocation}[i][j]$

Process	Maximum	Allocation	Need = Max - Alloc
P ₁	(2, 1, 1)	(1, 0, 1)	$(2 - 1, 1 - 0, 1 - 1) = (1, 1, 0)$
P ₂	(2, 2, 2)	(1, 1, 0)	$(2 - 1, 2 - 1, 2 - 0) = (1, 1, 2)$
P ₃	(3, 1, 2)	(1, 0, 1)	$(3 - 1, 1 - 0, 2 - 1) = (2, 1, 1)$

Part (b) & (c) — Safety State Determination & Safe Sequence

Step-by-Step Execution of Safety Algorithm:

- Let $Work = Available = (3, 2, 2)$
- Let $Finish = [False, False, False]$

Iteration 1:

- Check **P_1**: Is $Finish[P_1] == False$ and $Need[P_1] \leq Work$?

$$(1, 1, 0) \leq (3, 2, 2) \Rightarrow \text{TRUE}$$

- **P_1** can execute safely. Once finished, it releases its allocated resources:

$$Work = Work + Allocation[P_1] = (3, 2, 2) + (1, 0, 1) = (4, 2, 3)$$

- Set $Finish[P_1] = True$. **Sequence** = $\langle P_1 \rangle$.

Iteration 2:

- Check **P_2**: Is $Finish[P_2] == False$ and $Need[P_2] \leq Work$?

$$(1, 1, 2) \leq (4, 2, 3) \Rightarrow \text{TRUE}$$

- **P_2** executes and releases its resources:

$$Work = Work + Allocation[P_2] = (4, 2, 3) + (1, 1, 0) = (5, 3, 3)$$

- Set $Finish[P_2] = True$. **Sequence** = $\langle P_1, P_2 \rangle$.

Iteration 3:

- Check **P_3**: Is $Finish[P_3] == False$ and $Need[P_3] \leq Work$?

$$(2, 1, 1) \leq (5, 3, 3) \Rightarrow \text{TRUE}$$

- **P_3** executes and releases its resources:

$$Work = Work + Allocation[P_3] = (5, 3, 3) + (1, 0, 1) = (6, 3, 4)$$

- Set $Finish[P_3] = True$. **Sequence** = $\langle P_1, P_2, P_3 \rangle$.

Conclusion:

- All processes have $Finish[i] == True$.
- **The system is in a SAFE STATE.**
- **Safe Sequence:** $\langle P_1 \rightarrow P_2 \rightarrow P_3 \rangle$ (Also valid: $\langle P_1 \rightarrow P_3 \rightarrow P_2 \rangle$, since **P_3's** need $(2, 1, 1) \leq (4, 2, 3)$ as well).

Q5. Producer-Consumer Problem (Food Delivery Shared Queue)

SCENARIO:

A food-delivery application has a shared queue with a capacity of **4 orders**. Producer threads generate orders and consumer threads process them.

Part (a) — Problem of Simultaneous Queue Modification

If two producers simultaneously modify the shared queue without synchronization:

1. **Race Condition on Queue Pointer / Count:** Both producers read the same tail pointer (e.g., $tail = 2$) concurrently. Both write their new order to slot 2, **overwriting and permanently losing** one customer's food order.
2. **Queue Corruption:** When both increment the count ($count = count + 1$), the interleaving of low-level CPU assembly instructions causes count to increment by only 1 instead of 2, violating data structure integrity.

Part (b) — Role of mutex, empty, and full Semaphores

Semaphore	Initial Value	Operational Role
mutex	1	Ensures Mutual Exclusion . Guarantees only one thread modifies the shared queue at any time.
empty	4 (Capacity)	Counting semaphore tracking vacant slots . Prevents Buffer Overflow by blocking producers when the queue is full (<code>empty == 0</code>).
full	0	Counting semaphore tracking available orders . Prevents Buffer Underflow by blocking consumers when queue is empty (<code>full == 0</code>).

Part (c) — Sequence of Semaphore Operations by Producer

Before inserting an order into the queue:

1. `wait(empty);` → Decrements the vacant count. If queue has 0 empty slots, producer thread blocks until space is freed.
2. `wait(mutex);` → Acquires the lock on the shared queue; enters Critical Section.

After inserting the order into the queue:

3. `signal(mutex);` → Releases the lock on the shared queue; exits Critical Section.
4. `signal(full);` → Increments the occupied slot count; wakes up any sleeping consumer thread waiting on `full`.

Q6. Deadlock Detection in Four-Process System

SCENARIO:

Consider four processes (P_1, P_2, P_3, P_4) and three single-instance resources (R_1, R_2, R_3):

- **P_1**: Holds R_1 , Waiting For R_2
- **P_2**: Holds R_2 , Waiting For R_3
- **P_3**: Holds R_3 , Waiting For R_1
- **P_4**: Holds nothing, Waiting For R_2

Part (a) — Draw the Resource Allocation Graph (RAG)

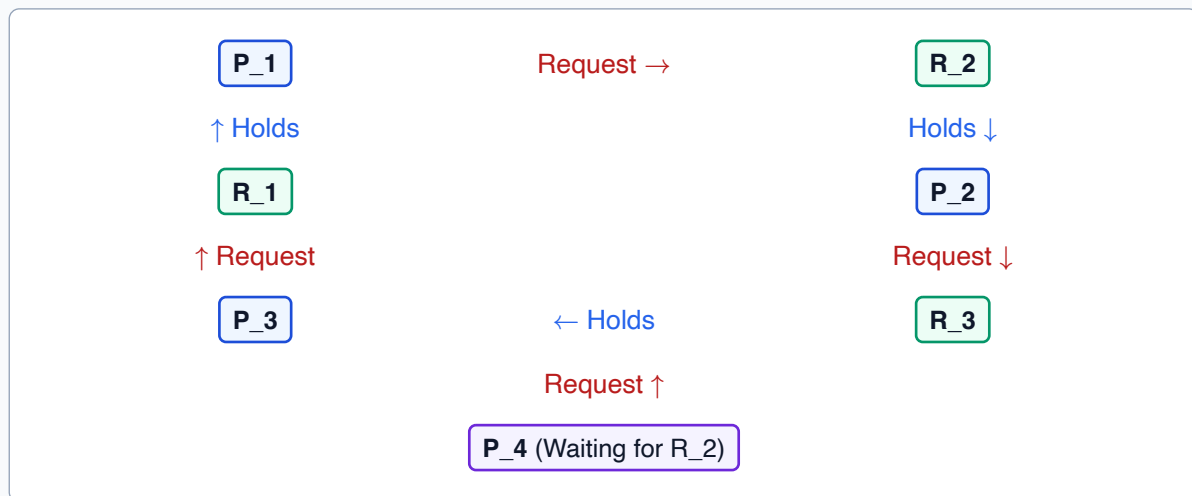


Figure 6.1: RAG with Cycle {P₁, R₂, P₂, R₃, P₃, R₁} and Dependent Process P₄

Part (b) — Identify Processes Involved in Deadlock

- **Processes strictly deadlocked in the cycle:** {P₁, P₂, P₃}
 - These three processes form an unbreakable circular dependency: P₁ → R₂ → P₂ → R₃ → P₃ → R₁ → P₁. None of them can ever proceed because each is waiting for a resource held by another process in the cycle.
- **Status of P₄:** Process P₄ is **blocked indefinitely (starved)** because it is waiting for R₂, which is held by the deadlocked process P₂. While P₄ is not part of the active circular dependency cycle itself, it will never execute unless the deadlock among {P₁, P₂, P₃} is resolved.

Part (c) — How Resource Ordering Prevents This Deadlock

1. **Total Resource Ordering Principle:** Assign an integer ranking to each resource:

$$F(R_1) = 1, \quad F(R_2) = 2, \quad F(R_3) = 3$$
2. **Enforcement Rule:** Every process must request resources in strictly monotonically increasing order. A process holding resource R_i can only request R_j if $F(R_j) > F(R_i)$.
3. **Prevention Mechanism:** In the current deadlock:
 - P₁ holds R₁ (F = 1) and requests R₂ (F = 2) → **Allowed** (2 > 1).
 - P₂ holds R₂ (F = 2) and requests R₃ (F = 3) → **Allowed** (3 > 2).
 - P₃ holds R₃ (F = 3) and requests R₁ (F = 1) → **DISALLOWED** because 1 < 3.
4. Process P₃ must release R₃ before it can request R₁, or it must request R₁ before R₃. This eliminates the circular wait condition entirely.

UNIT IV: MEMORY MANAGEMENT

Continuous Concurrent Evaluation (CCE2) — Comprehensive Worked Solutions

Course Pattern: TE IT - 2024 Pattern (SPPU)
Document Type: CCE2 Model Answer Solutions

Academic Term: Semester V (2026)
Course Code: PCC-301-ITT

NOTE:

This document provides exhaustive, publication-grade model answers and worked solutions for all questions in **Unit IV (Memory Management)** from the official CCE2 question bank. Step-by-step frame contents after every page reference, paging address calculations, partition allocation traces, Buddy System calculations, and thrashing analyses are rigorously detailed.

Q1. Page Replacement (Reference String: 1, 2, 3, 1, 4, 2, 5, 1)

SCENARIO:

A process has **3 available physical frames** and generates the following page-reference string:

1, 2, 3, 1, 4, 2, 5, 1

Part (a) — FIFO Page Replacement

FIFO (First-In, First-Out) replaces the page that has been in memory for the longest duration.

Ref String	1	2	3	1	4	2	5	1
Frame 1	1	1	1	1	4	4	4	1
Frame 2	-	2	2	2	2	2	5	5
Frame 3	-	-	3	3	3	3	3	3
Status	Fault	Fault	Fault	Hit	Fault	Hit	Fault	Fault

• Step-by-step trace:

- Ref 1: Miss → Load 1 in Frame 1 (**Page Fault 1**)
- Ref 2: Miss → Load 2 in Frame 2 (**Page Fault 2**)
- Ref 3: Miss → Load 3 in Frame 3 (**Page Fault 3**)
- Ref 1: Hit → Page 1 already in Frame 1 (**Hit**)
- Ref 4: Miss → Oldest is Page 1 in Frame 1; replace 1 with 4 (**Page Fault 4**)
- Ref 2: Hit → Page 2 already in Frame 2 (**Hit**)
- Ref 5: Miss → Oldest is Page 2 in Frame 2; replace 2 with 5 (**Page Fault 5**)
- Ref 1: Miss → Oldest is Page 3 in Frame 3; replace 3 with 1 (**Page Fault 6**)

Total Page Faults (FIFO) = 6 (Hits = 2, Fault Rate = $\frac{6}{8} = 75\%$)

Part (b) — LRU Page Replacement

LRU (Least Recently Used) replaces the page that has not been referenced for the longest period of time.

Ref String	1	2	3	1	4	2	5	1
Frame 1	1	1	1	1	1	1	5	5
Frame 2	-	2	2	2	4	4	4	1
Frame 3	-	-	3	3	3	2	2	2
Status	Fault	Fault	Fault	Hit	Fault	Fault	Fault	Fault

- **Step-by-step trace:**

1. Ref 1, 2, 3: Load pages into empty frames (**Page Faults 1, 2, 3**).
2. Ref 1: Hit in Frame 1 (**Hit**). Recent order of pages (most → least): 1, 3, 2.
3. Ref 4: Miss → Least recently used is Page 2; replace 2 with 4 (**Page Fault 4**). Recent order: 4, 1, 3.
4. Ref 2: Miss → Least recently used is Page 3; replace 3 with 2 (**Page Fault 5**). Recent order: 2, 4, 1.
5. Ref 5: Miss → Least recently used is Page 1; replace 1 with 5 (**Page Fault 6**). Recent order: 5, 2, 4.
6. Ref 1: Miss → Least recently used is Page 4; replace 4 with 1 (**Page Fault 7**). Recent order: 1, 5, 2.

Total Page Faults (LRU) = 7 (Hits = 1, Fault Rate = $\frac{7}{8} = 87.5\%$)

Part (c) — Algorithm Comparison for this Reference String

- **FIFO performs better** for this specific reference string:
 - **FIFO Page Faults: 6**
 - **LRU Page Faults: 7**
- **Reasoning:** Although LRU is theoretically superior to FIFO in capturing temporal locality on average workloads, on certain short or cyclic reference strings (such as this one), FIFO incurs fewer page faults because LRU's recent hit on Page 1 retained it just long enough to cause eviction of Page 2, which was needed immediately afterwards.

Q2. Paging and Address Translation

SCENARIO:

A system has a logical address space of **32 KB** and a page size of **4 KB**. Physical memory is **16 KB**.

Part (a) — Calculate the Number of Pages in Logical Address Space

Given:

- Logical Address Space (LAS) = 32 KB = 32×1024 Bytes = 32,768 Bytes = 2^{15} Bytes
- Page Size (P) = 4 KB = 4×1024 Bytes = 4096 Bytes = 2^{12} Bytes

Calculation:

$$\text{Number of Pages} = \frac{\text{Logical Address Space}}{\text{Page Size}} = \frac{32 \text{ KB}}{4 \text{ KB}} = 8 \text{ pages}$$

- Logical page numbers range from **Page 0 to Page 7**.
- Logical Address format: 15 bits total → 3 bits for Page Number ($2^3 = 8$), 12 bits for Page Offset ($2^{12} = 4096$).

Part (b) — Calculate the Number of Frames in Physical Memory

Given:

- Physical Memory (PAS) = 16 KB = 16×1024 Bytes = 16,384 Bytes = 2^{14} Bytes
- Frame Size = Page Size = 4 KB = 2^{12} Bytes

Calculation:

$$\text{Number of Frames} = \frac{\text{Physical Memory Size}}{\text{Frame Size}} = \frac{16 \text{ KB}}{4 \text{ KB}} = 4 \text{ frames}$$

- Physical frame numbers range from **Frame 0 to Frame 3**.
- Physical Address format: 14 bits total → 2 bits for Frame Number ($2^2 = 4$), 12 bits for Offset.

Part (c) — How Paging Eliminates External Fragmentation

1. **Definition of External Fragmentation:** External fragmentation occurs when total free memory is sufficient to satisfy a process allocation request, but the memory is divided into small, non-contiguous blocks, preventing allocation.
2. **Mechanism of Elimination in Paging:**
 - Paging partitions physical memory into **fixed-size blocks called frames**.
 - A process's logical address space is divided into chunks of the exact same size called **pages**.
 - Any available physical frame anywhere in RAM can be allocated to any logical page of a process via the **Page Table**.
 - Because memory allocation is done in discrete, uniform-sized frame units, **no space between allocated blocks ever remains unallocated or unusable**.
 - Contiguity in physical memory is **never required**; non-contiguous physical frames appear completely contiguous to the CPU through the hardware MMU. Thus, **external fragmentation is completely eliminated**.
3. **(Note on Internal Fragmentation):** While external fragmentation is eliminated, paging can still suffer from minimal **internal fragmentation** in the process's final page (on average, $\frac{1}{2}$ page size = 2 KB).

Q3. Page Replacement (Reference String: 7, 0, 1, 2, 0, 3, 0, 4)

SCENARIO:

A system has **3 physical frames** and the page-reference string is:

7, 0, 1, 2, 0, 3, 0, 4

Show the contents of the frames after each page reference.

Part (a) — FIFO Page Replacement

Ref String	7	0	1	2	0	3	0	4
Frame 1	7	7	7	2	2	2	0	0
Frame 2	-	0	0	0	0	3	3	4
Frame 3	-	-	1	1	1	1	1	1
Hit / Fault	Fault	Fault	Fault	Fault	Hit	Fault	Fault	Fault

• Detailed Trace:

- Ref 7: Miss → Frame 1 = [7] (**Fault 1**)
- Ref 0: Miss → Frame 1 = [7], Frame 2 = [0] (**Fault 2**)
- Ref 1: Miss → Frame 1 = [7], Frame 2 = [0], Frame 3 = [1] (**Fault 3**)
- Ref 2: Miss → Oldest is 7; replace 7 in Frame 1 with 2 → [2, 0, 1] (**Fault 4**)
- Ref 0: Hit → Page 0 already present in Frame 2 (**Hit 1**)
- Ref 3: Miss → Oldest is 0; replace 0 in Frame 2 with 3 → [2, 3, 1] (**Fault 5**)
- Ref 0: Miss → Oldest is 1; replace 1 in Frame 3 with 0 → [2, 3, 0] (**Fault 6**)
- Ref 4: Miss → Oldest is 2; replace 2 in Frame 1 with 4 → [4, 3, 0] (**Fault 7**)

Total Page Faults (FIFO) = 7 (Hits = 1)

Part (b) — LRU Page Replacement

Ref String	7	0	1	2	0	3	0	4
Frame 1	7	7	7	2	2	2	2	4
Frame 2	-	0	0	0	0	0	0	0
Frame 3	-	-	1	1	1	3	3	3
Hit / Fault	Fault	Fault	Fault	Fault	Hit	Fault	Hit	Fault

• Detailed Trace:

- Ref 7, 0, 1: Loaded into empty frames → [7, 0, 1] (**Faults 1, 2, 3**)
- Ref 2: Miss → History: 7 was referenced at t_1 , 0 at t_2 , 1 at t_3 . Least recently used is 7; replace 7 in Frame 1 with 2 → [2, 0, 1] (**Fault 4**)
- Ref 0: Hit → Page 0 accessed. Recent usage order: 0 (most recent), 2, 1 (least recent) (**Hit 1**)
- Ref 3: Miss → Least recently used is 1; replace 1 in Frame 3 with 3 → [2, 0, 3] (**Fault 5**)
- Ref 0: Hit → Page 0 accessed. Recent usage order: 0 (most recent), 3, 2 (least recent) (**Hit 2**)
- Ref 4: Miss → Least recently used is 2; replace 2 in Frame 1 with 4 → [4, 0, 3] (**Fault 6**)

Total Page Faults (LRU) = 6 (Hits = 2)

Comparison: LRU outperforms FIFO with **6 page faults vs 7 page faults**.

Q4. Memory Allocation (First Fit)

SCENARIO:

A system has the following free memory blocks (ordered left to right):

100 KB, 500 KB, 200 KB, 300 KB, 600 KB

The following processes arrive in order:

$P_1 = 212$ KB, $P_2 = 417$ KB, $P_3 = 112$ KB, $P_4 = 426$ KB

Part (a) & (b) — Allocate using First Fit and Identify Unused Memory

First Fit Algorithm Rule: Scan the free memory blocks from the beginning (left to right) and allocate the **very first block** that is large enough (Block Size $\geq$ Process Size).

Process	Size Required	Block Allocated	Block Original Size	Unused Memory
P1	212 KB	Block 2	500 KB	500 - 212 = 288 KB
P2	417 KB	Block 5	600 KB	600 - 417 = 183 KB
P3	112 KB	Block 3	200 KB	200 - 112 = 88 KB
P4	426 KB	CANNOT ALLOCATE	No remaining block $\geq$ 426 KB	Must Wait

Allocation Steps Breakdown:

- Process P1 (212 KB):** Check Block 1 (100 KB) → Too small. Check Block 2 (500 KB) → Fits! Allocated to **Block 2**.
- Process P2 (417 KB):** Check Block 1 (100 KB) → Too small. Block 2 is occupied. Check Block 3 (200 KB) → Too small. Check Block 4 (300 KB) → Too small. Check Block 5 (600 KB) → Fits! Allocated to **Block 5**.
- Process P3 (112 KB):** Check Block 1 (100 KB) → Too small. Block 2 occupied. Check Block 3 (200 KB) → Fits! Allocated to **Block 3**.
- Process P4 (426 KB):** Blocks remaining free are Block 1 (100 KB) and Block 4 (300 KB). Neither can fit 426 KB. Hence, **P4 cannot be allocated and must wait**.

Unused Memory Caused by Allocation:

- Unused space inside occupied partitions: 288 KB + 183 KB + 88 KB = **559 KB**
- Completely unallocated partitions: Block 1 (100 KB) + Block 4 (300 KB) = **400 KB**
- Total unused physical memory** = 559 KB + 400 KB = **959 KB**

Part (c) — Internal vs External Fragmentation with Reference to this Example

Fragmentation	Theoretical Definition	Demonstration in this Example
Internal Fragmentation	Memory allocated to a process is larger than requested; the unused wasted space lies inside the allocated partition and cannot be used by other processes.	In Fixed Partitioning: - Block 2 (500 KB) given to P1 (212 KB) leaves 288 KB internal fragmentation. - Block 5 (600 KB) given to P2 (417 KB) leaves 183 KB internal fragmentation. - Block 3 (200 KB) given to P3 (112 KB) leaves 88 KB internal fragmentation.
External Fragmentation	Total unused free memory across the system is sufficient to satisfy a process request, but the storage is fragmented into non-contiguous blocks, preventing allocation.	Process P4 requires 426 KB . - Remaining free unallocated blocks: Block 1 (100 KB) + Block 4 (300 KB) = 400 KB. - If dynamic partitioning is considered with internal remnants ($288 + 183 + 88 = 559$ KB), total available free memory is $400 + 559 = 959$ KB, which is far greater than 426 KB. Yet P4 cannot run because the free space is non-contiguous.

Q7. Virtual Memory Architecture (Physical Memory 32 KB, Page 4 KB, Virtual 64 KB)

SCENARIO:

A computer running Google Chrome processes/tabs has:

- Physical Memory = **32 KB**
- Page Size = **4 KB**
- Virtual Address Space = **64 KB**

Part (a) — Calculate the Number of Pages in Virtual Address Space

$$\text{Total Virtual Pages} = \frac{\text{Virtual Address Space}}{\text{Page Size}} = \frac{64 \text{ KB}}{4 \text{ KB}} = 16 \text{ pages}$$

- Each Chrome process/tab has a 64 KB address space containing 16 virtual pages (Page 0 through Page 15).
- Virtual Address requires 16 bits total: $\log_2(64 \times 1024) = 16$ bits.
- Virtual Page Number (p) = 4 bits ($2^4 = 16$), Page Offset (d) = 12 bits ($2^{12} = 4096 = 4 \text{ KB}$).

Part (b) — Calculate Number of Frames Available in Physical Memory

$$\text{Total Physical Frames} = \frac{\text{Physical Memory Size}}{\text{Page Size}} = \frac{32 \text{ KB}}{4 \text{ KB}} = 8 \text{ frames}$$

- Physical memory has 8 hardware page frames (Frame 0 through Frame 7).
- Physical Address requires 15 bits total: $\log_2(32 \times 1024) = 15$ bits.
- Frame Number (f) = 3 bits ($2^3 = 8$), Offset (d) = 12 bits.

Part (c) — How 64-KB Virtual Space is Managed using 32-KB Physical Memory

Through the mechanism of **Virtual Memory with Demand Paging**:

1. **Non-Resident Pages on Secondary Storage:** The entire 64 KB virtual memory of a process does not reside simultaneously in physical RAM. Instead, all 16 pages are stored on the swap space/backing store of the hard drive.
2. **Page Table with Valid/Invalid Bits:** Each page table entry includes a **Valid-Invalid bit**:
 - Bit = 1 (Valid): The virtual page currently resides in one of the 8 physical frames.
 - Bit = 0 (Invalid): The virtual page resides on the disk backing store.
3. **Demand Paging & Page Fault Handling:**
 - A page is brought into physical memory **only when referenced** (demanded) by the CPU.
 - If the CPU accesses a page with bit = 0, the MMU generates a hardware trap called a **Page Fault**.
 - The OS traps to kernel mode, selects a victim page in physical memory using a page replacement algorithm (e.g., LRU), swaps the victim out to disk, and reads the requested virtual page into the vacated frame.
4. **Outcome:** Google Chrome can seamlessly execute processes that require more memory than physically installed, because only the currently active working set of pages occupies physical RAM.

Q5. Browser Application Page Replacement (Ref: 2, 3, 2, 1, 5, 2, 4, 5, 3)

SCENARIO:

A browser application has **3 frames available**. Page-reference string:

2, 3, 2, 1, 5, 2, 4, 5, 3

Part (a) — FIFO Page Replacement

Ref String	2	3	2	1	5	2	4	5	3
Frame 1	2	2	2	2	5	5	5	5	3
Frame 2	-	3	3	3	3	2	2	2	2
Frame 3	-	-	-	1	1	1	4	4	4
Hit / Fault	Fault	Fault	Hit	Fault	Fault	Fault	Fault	Hit	Fault

• **Trace:**

- Ref 2, 3: Loaded into Frame 1, Frame 2 (**Faults 1, 2**)
- Ref 2: Hit in Frame 1 (**Hit 1**)
- Ref 1: Loaded into Frame 3 (**Fault 3**) → Frames: [2, 3, 1]
- Ref 5: Miss → Oldest is 2; replace with 5 → [5, 3, 1] (**Fault 4**)
- Ref 2: Miss → Oldest is 3; replace with 2 → [5, 2, 1] (**Fault 5**)
- Ref 4: Miss → Oldest is 1; replace with 4 → [5, 2, 4] (**Fault 6**)
- Ref 5: Hit in Frame 1 (**Hit 2**)
- Ref 3: Miss → Oldest is 5; replace with 3 → [3, 2, 4] (**Fault 7**)

Total Page Faults (FIFO) = 7 (Hits = 2)

Part (b) — LRU Page Replacement

Ref String	2	3	2	1	5	2	4	5	3
Frame 1	2	2	2	2	2	2	2	2	3
Frame 2	-	3	3	3	5	5	4	4	4
Frame 3	-	-	-	1	1	1	1	5	5
Hit / Fault	Fault	Fault	Hit	Fault	Fault	Hit	Fault	Hit	Fault

• **Trace:**

- Ref 2, 3: Loaded → [2, 3] (**Faults 1, 2**)
- Ref 2: Hit → Recent order: 2 (most recent), 3 (**Hit 1**)
- Ref 1: Loaded into Frame 3 → [2, 3, 1] (**Fault 3**). Recent order: 1, 2, 3
- Ref 5: Miss → LRU page is 3; replace 3 with 5 → [2, 5, 1] (**Fault 4**). Recent order: 5, 1, 2
- Ref 2: Hit in Frame 1 (**Hit 2**). Recent order: 2, 5, 1
- Ref 4: Miss → LRU page is 1; replace 1 with 4 → [2, 5, 4] (**Fault 5**). Recent order: 4, 2, 5
- Ref 5: Hit in Frame 2 (**Hit 3**). Recent order: 5, 4, 2
- Ref 3: Miss → LRU page is 2; replace 2 with 3 → [3, 5, 4] (**Fault 6**).

Total Page Faults (LRU) = 6 (Hits = 3)

Part (c) — Conclusion

LRU gives fewer page faults (6 page faults) compared to FIFO (7 page faults).

Q9. Program Relocation and Address Translation

SCENARIO:

A program is compiled assuming it starts at address **1000**. During execution, the OS loads it at address **5000**. If the program accesses logical address **1250**, calculate the corresponding physical address using relocation. Explain why relocation is required.

Part (a) — Calculate Corresponding Physical Address

Method 1 — Base-Relative Offset Formula:

- Compiled Base Address (B_{compiled}) = 1000
- Logical Address referenced (L) = 1250
- Relative Offset within Program (Offset) = $L - B_{\text{compiled}} = 1250 - 1000 = 250$
- Loaded Physical Base Address (B_{physical}) = 5000
- Physical Address = $B_{\text{physical}} + \text{Offset} = 5000 + 250 = 5250$

Method 2 — Relocation Delta / Relocation Register:

- Relocation Delta (Δ) = Loaded Base – Compiled Base = $5000 - 1000 = +4000$
- Physical Address = Logical Address + $\Delta = 1250 + 4000 = 5250$

Answer: The corresponding physical address is 5250.

Part (b) — Why Relocation is Required in Operating Systems

1. **Multiprogramming & Dynamic Loading:** In modern operating systems, multiple programs reside concurrently in main memory. The operating system cannot predict in advance which physical memory range will be free when a user launches an application. Relocation allows a program to execute at **any physical memory location**.
2. **Swapping and Compaction:** When memory is tight, an idle process may be swapped out to disk. When swapped back into RAM later, its previous physical addresses may already be occupied by other processes. Dynamic relocation allows the OS to load the program into a completely different physical address space seamlessly.
3. **Memory Protection and Isolation:** Programs generate logical addresses independent of actual hardware memory. The relocation register combined with a limit register ensures that a process cannot access another process's memory space or corrupt OS kernel code.

Q6. Virtual Memory and Thrashing (Google Chrome Case Study)

SCENARIO:

Google Chrome is running with many tabs open. The system begins spending most of its time transferring pages between disk and main memory, resulting in very poor performance.

Part (a) — Identify the Memory-Management Problem

The problem is **THRASHING**.

Part (b) — Explain Why Thrashing Occurs

1. **High Degree of Multiprogramming:** As the user opens many Chrome tabs, each tab requires a specific set of active pages in memory to execute, known as its **Working Set**.
2. **Working Set Sum Exceeds Physical RAM:** When the sum of the working set sizes of all active processes exceeds total available physical frames:

$$\sum |WS_i| > \text{Total Physical Frames}$$

3. **Cascading Page Fault Loop:** Processes constantly experience page faults. The OS attempts to bring in demanded pages by evicting other pages. However, the evicted pages are immediately needed again by their respective processes, causing immediate subsequent page faults.
4. **I/O Dominance:** The CPU scheduler notices CPU utilization dropping because all processes are queued waiting for the paging disk. Misinterpreting this as low workload, the OS may attempt to increase multiprogramming, exacerbating the problem. Consequently, the disk queue fills, swapping throughput peaks, and system throughput plummets to near zero.

Part (c) — What is Demand Paging?

Demand Paging is a virtual memory management technique where pages of a process are loaded into physical memory (RAM) **only when they are demanded (referenced) during execution**, rather than pre-loading the entire program into memory at process creation.

- It relies on a **lazy swapper / pager**.
- Reduces initial loading time, minimizes memory consumption, and allows programs larger than physical memory to run.

Part (d) — Two Ways to Reduce or Prevent Thrashing

1. **Working-Set Model (Working Set Strategy):**
 - The OS monitors the **Working Set** ($WS_{i(\Delta)}$) of each process, defined as the set of pages referenced in the most recent Δ time window.
 - The OS allocates each process enough frames to fit its entire working set. If total working set demand exceeds available frames, the OS **suspends (swaps out) one entire process**, freeing its frames for the remaining processes until thrashing subsides.
2. **Page-Fault Frequency (PFF) Control:**
 - Establish an acceptable **Upper Bound** and **Lower Bound** for the page-fault frequency of each process.
 - If a process's page-fault rate exceeds the upper threshold, allocate it additional physical frames.
 - If the page-fault rate drops below the lower threshold, reclaim unused frames.
 - If overall page faults remain critically high and no frames are free, swap out a process.
3. **(Practical System Action):** Closing unused background Chrome tabs, terminating non-essential applications, or installing additional physical RAM.

Q8. Buddy System Memory Allocation

SCENARIO:

An operating system has **1024 MB physical memory**. The Buddy System allocates memory strictly in powers of 2.

- Process A requests **100 MB**.
- Process B requests **200 MB**.

Part (a) — Block Size Allocated for 100 MB Request

In the binary Buddy System, memory requests are fulfilled by finding the smallest power of 2 that is greater than or equal to the requested size S :

$$\text{Allocated Block Size} = 2^k \quad \text{such that} \quad 2^{k-1} < S \leq 2^k$$

For a request of $S = 100$ MB:

- $2^6 = 64$ MB < 100 MB
- $2^7 = 128$ MB ≥ 100 MB

Splitting Process from 1024 MB:

1. Split 1024 MB into two buddies of 512 MB.
2. Split one 512 MB block into two buddies of 256 MB.
3. Split one 256 MB block into two buddies of 128 MB.
4. Allocate one 128 MB block to the process.

Answer: A block size of 128 MB will be allocated.

Part (b) — Internal Fragmentation for 100 MB Request

Internal fragmentation is the allocated block size minus the requested memory size:

$$\text{Internal Fragmentation} = \text{Allocated Size} - \text{Requested Size}$$

$$\text{Internal Fragmentation} = 128 \text{ MB} - 100 \text{ MB} = \mathbf{28 \text{ MB}}$$

Part (c) — Block Size Allocated for 200 MB Request

For a request of $S = 200$ MB:

- $2^7 = 128$ MB < 200 MB
- $2^8 = 256$ MB ≥ 200 MB

The OS takes the available 256 MB block created during the earlier split and allocates it directly to Process B.

- **Allocated Block Size:** 256 MB
- **Internal Fragmentation for Process B:** $256 \text{ MB} - 200 \text{ MB} = \mathbf{56 \text{ MB}}$